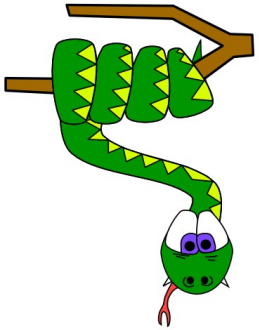


python™

Comments

comments

- makes the program more readable
- helps us remember why certain blocks of code were written.
- comments can also be used to ignore some code while testing other blocks of code.
- In Python, we use the hash symbol **#** to write a single-line comment.

```
# printing a string
print('Hello world')
```

Output

```
Hello world
```

- comment line is ignored by the Python interpreter.

comments

- Everything that comes after # is ignored.
- So, we can also write the above program in a single line as:

```
print('Hello world') #printing a string
```

- Multi-Line Comments in Python
- Python doesn't offer a separate way to write multiline comments. However, there are other ways to get around this issue.

```
# it is a
# multiline
# comment
```

comments

- **String Literals for Multi-line Comments**
- Python interpreter ignores the string literals that are not assigned to a variable.

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

- As long as the string is not assigned to a variable, Python will read the code, but then ignore it, and you have made a multiline comment.

comments

- **Documentation Strings**
- string literals that appear right after the definition of a function, method, class, or module.

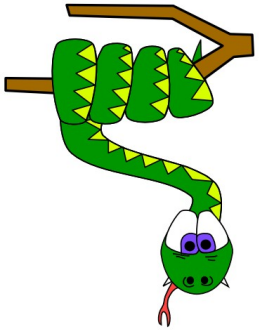
- Example

```
def add(a,b):
        """This function add two values"""
        c=a+b
        print(c)
```

- The strings are accessible in the **_ _ doc _ _** attribute of an object. print docstring as shown here:

print(add._ _doc_ _)

This function add two values



Reading input, Print output

Python input()

- Python has an **input()** function which lets you ask a user for some text input.
- Call this function to tell the program to stop and wait for the user to key in the data.
- In Python 2, you have a built-in function raw_input(),

whereas in Python 3, you have input()

- syntax :

input([prompt])

- Example

```
>>> num = input('Enter a number: ')
Enter a number: 10
>>> num
'10'
```

Python input()

- Here, we can see that the entered value 10 is a string, not a number.
- To convert this into a number we can use int() or float() functions.

```
>>> int('10')
10
>>> float('10')
10.0
```

- This can be done in the same line while reading input

```
>>> num = int(input('Enter a number: '))
Enter a number: 10
>>> num
10
```

Python print()

The print() function prints the given object to the standard output device (screen) or to the text stream file.

- syntax :

```
print(object(s), sep=separator, end=end, file=file, flush=flush)
```

Parameter Values

object(s) - Any object, and as many as you like. Will be converted to string before printed

sep='separator' - Optional. Specify how to separate the objects, if there is more than one. Default is ' '

end='end' - Optional. Specify what to print at the end. Default is '\n' (line feed)

Python print()

file - Optional. An object with a write method. Default is sys.stdout

flush - Optional. A Boolean, specifying if the output is flushed (True) or buffered (False). Default is False

1. Python print()

```
print("Python is fun.")
a = 5

# Two objects are passed
print("a =", a)

b = a
# Three objects are passed
print('a =', a, '= b')
```

Output:

```
Python is fun.
a = 5
a = 5 = b
```

1. Python print()

To suppress or change the line ending, use the **end=ending keyword** argument. For example:

```
print("The values are", x, y, z, end='') # Suppress the newline
```

To redirect the output to a fle, use the file=outfile keyword argument. For example:

```
print("The values are", x, y, z, file=f) # Redirect to fle object f
```

To change the separator character between items, use the sep=sepchr keyword argument. For example:

```
print("The values are", x, y, z, sep=',') # Put commas between the values
```

2. using format()

Formatting output using the format method :

- The **format()** method was added in Python(2.6). The format method of strings requires more manual effort. Users use {} to mark where a variable will be substituted and can provide detailed formatting directives

```
# using format() method
print('I love {}  {}".format('Python', 'program'))

# using format() method and refering
# a position of the object
print('{0} and {1}'.format('C', 'C++'))

print('{1} and {0}'.format('C', 'C++'))
```

Output:

I love Python program
C and C++
C++ and C

Add.py

```
# Store input numbers
num1 = input('Enter first number: ')
num2 = input('Enter second number: ')

# Add two numbers
sum = float(num1) + float(num2)

# Display the sum
print('The sum of {0} and {1} is {2}'.format(num1, num2, sum))
```

```
The sum of 1.5 and 6.3 is 7.8
```

3. using modulo (%)

Sometimes user often wants more control the formatting of output than simply printing space-separated values

Formatting output using String modulo operator(%) :

- On the left side of the % operator, <format_string> is a string containing one or more conversion specifiers. The <values> on the right side get inserted into <format_string> in place of the conversion specifiers.

```
print("%d %s cost %.2f  rupees" % (6, 'bananas', 37.5))
```

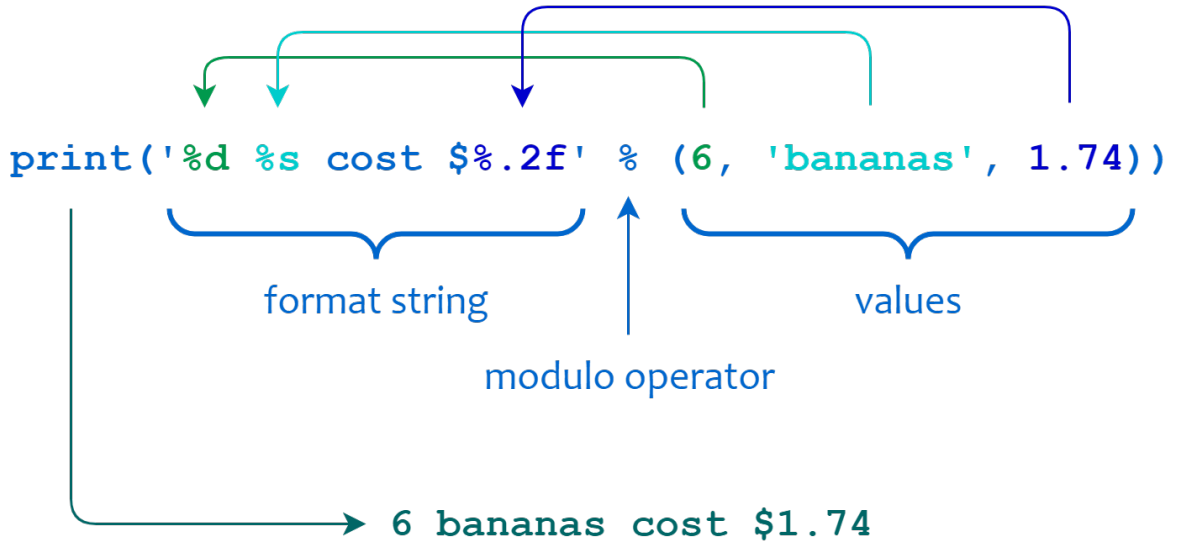
```
Output :
6 bananas cost 37.5 rupees
```

3. using modulo (%)

print('%d %s cost $%.2f' % (6, 'bananas', 1.74))

Output :

```
6 bananas cost $1.74
```



4. f-strings

f-strings (formatted string literals) were introduced in Python 3.6 to make string formatting easier and more readable.

They allow variables and expressions to be directly embedded inside strings using curly braces {}.

```
# using f' sring
name = "Emily"
age = 20
print(f"My name is {name} and I am {age} years old")
```

Output

```
My name is Emily and I am 20 years old
```

An f-string is created by adding **f** before the string and placing variables or expressions inside {}.